**PHENIKAA UNIVERSITY**

**PHENIKAA SCHOOL OF COMPUTING**

□□□□□


**COURSE: SOFTWARE ARCHITECTURE**

**TOPIC:**

# QR CODE-BASE IN-TABLE FOOD ORDERING SYSTEM

**REPORT: Lab 8 - Deployment View & Quality Attribute Analysis (ATAM)**

**Code Course** : CSE703110-1-3-25

**Class** : N01

**Instructor** : THS. Vũ Quang Dũng

**Group** : 1

**Members** : 1. Nguyễn Đình Trung - 23010701

2. Vũ Anh Nam - 23010886

3. Đặng Minh Hoàng - 23010445

4. Nguyễn Phan Quang Minh - 23010530

## Mục Lục

**1. Abstract** ..... 3

**2. Objectives** ..... 3

**3. System Overview and Architectural Context** ..... 3

3.1 Background ..... 3

**4. Deployment Diagram** ..... 4

4.1 Deployment Environment ..... 4

4.2 Communication Between Nodes ..... 4

**5. Architecture Trade Off Analysis Method Overview** ..... 4

**6. ATAM Analysis for Scalability** ..... 5

6.1 Scalability Scenario ..... 5

6.2 Monolithic Architecture and Scalability ..... 5

6.3 Microservices Architecture and Scalability ..... 5

6.4 Scalability Decision in the System ..... 5

**7. ATAM Analysis for Availability** ..... 6

7.1 Availability Scenario ..... 6

7.2 Monolithic Architecture and Availability ..... 6

7.3 Microservices Architecture and Availability ..... 6

7.4 Availability Decision in the System ..... 6

**8. Discussion** ..... 7

**9. Conclusion** ..... 7

## 1. Abstract

Lab 8 focuses on analyzing the software architecture of the QR Code-Based In-Table Food Ordering System (FOODASH) from the perspectives of physical infrastructure deployment and formal quality attribute verification. As a tableside hospitality platform scaling to handle sudden operational traffic spikes during peak restaurant dining hours, mapping the physical layout of software components onto execution environments is critical. This laboratory document introduces a comprehensive UML Deployment Diagram utilizing a containerized architecture managed via Docker. Furthermore, a simplified Architecture Trade-Off Analysis Method (ATAM) is performed to evaluate the current hybrid Modular Monolithic design against pure monolith and fully distributed microservices topologies, focusing on the trade-offs between Scalability and Availability.

## 2. Objectives

The strategic technical objectives established for this laboratory study include:

- Physical Mapping: Constructing a professional UML Deployment Diagram representing the runtime node infrastructure of the FOODASH ecosystem.
- Containerization Modeling: Defining the network communication, ports, and execution isolation layers of the systems using container concepts.
- ATAM Execution: Applying the formal Architecture Trade-Off Analysis Method to analyze structural decisions regarding system scalability.
- Availability Analysis: Evaluating failure domain limits and system failover strategies to ensure continuous operational uptime during severe server or network stress events.

## 3. System Overview and Architectural Context

### 3.1 Background

The FOODASH platform allows restaurant guests to instantly scan tableside QR codes, browse real-time digital menus, compile multi-item food carts, and initiate banking transactions via the integrated VNPay gateway wrapper. Concurrently, the platform updates kitchen display queues and notifies service staff via webhooks. The underlying architecture operates as a hybrid Modular Monolith packaged as a Node.js/Express core application layer backed by an in-memory Redis cache engine and a relational PostgreSQL database instance. To support peak transactional waves, non-blocking

asynchronous processing pipelines are routed through a standalone RabbitMQ message broker.

## 4. Deployment Diagram

### 4.1 Deployment Environment

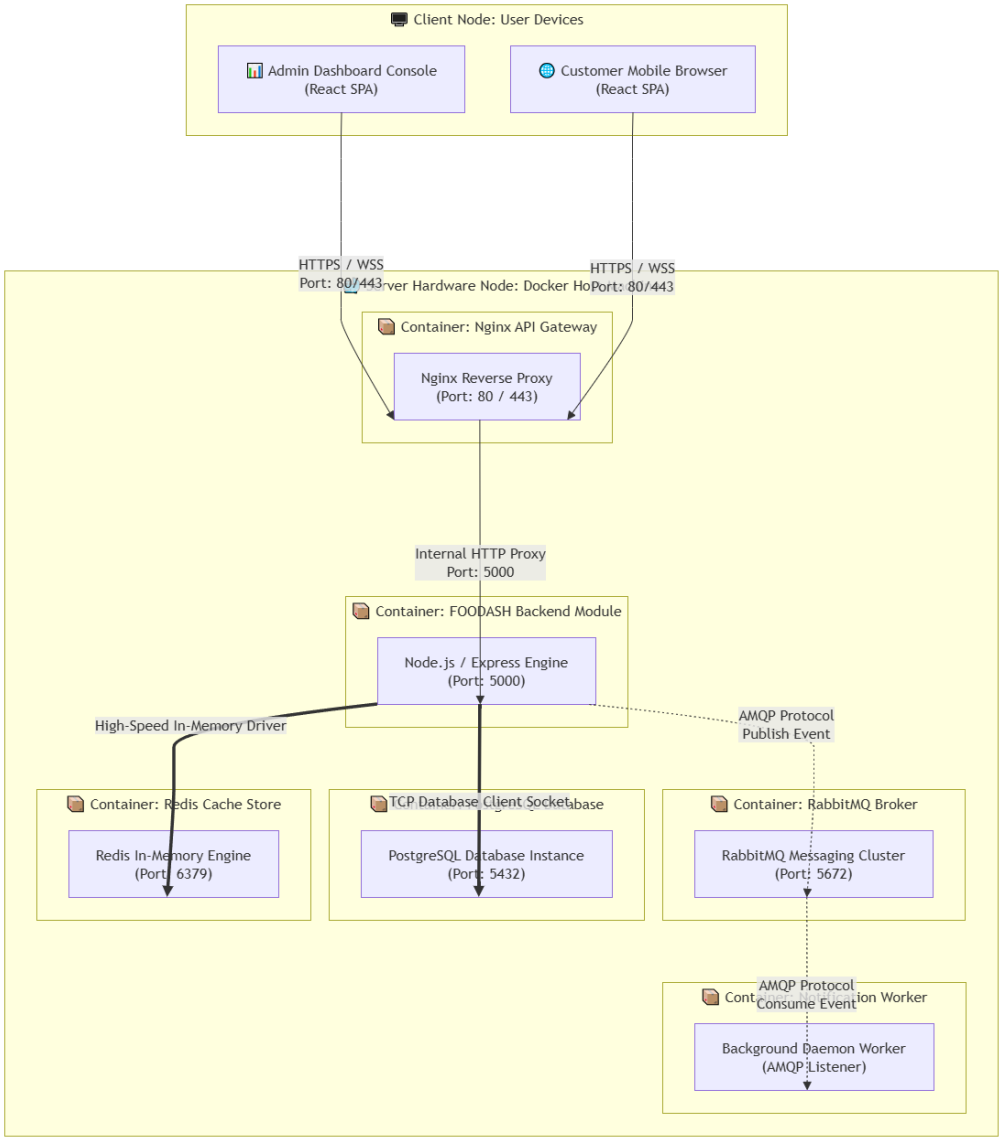
The physical deployment environment of FOODASH utilizes a lightweight, containerized architecture managed via Docker, facilitating seamless environment synchronization and cloud-native orchestration. The multi-node deployment model maps application topologies across four specialized container layers:

- Edge Proxy Node (Nginx API Gateway): Exposed directly to public client devices on port 80/443. It manages SSL termination, Cross-Origin Resource Sharing (CORS) configurations, and proxies inbound requests to the application servers.

- Application Runtime Node (FOODASH Backend Module): Hosts the monolithic Node.js and Express engine on port 5000. It encapsulates core application layers, executing business rules for menu management, live ordering, and checkout billing.

- Asynchronous Integration Nodes (RabbitMQ Broker & Worker): Consists of the central RabbitMQ cluster container (port 5672) and detached background daemon service workers that asynchronously pop messages to update kitchen monitors and process push notifications.

- Data Storage Persistence Node (PostgreSQL & Redis Clusters): A private data tier hosting the primary PostgreSQL relational database (port 5432) for ACID financial compliance, alongside the high-speed Redis cache store (port 6379) for caching high-frequency read menus.




### 4.2 Communication Between Nodes

Communication paths across execution nodes are strictly standardized using production-grade protocols. Client applications trigger actions over secure public HTTPS REST API connections hitting the Nginx edge. Internally, Nginx forwards HTTP request payloads to the internal Node.js container cluster over a private network mesh. Inter-module communication and external worker coordination rely on advanced, non-blocking AMQP bindings via RabbitMQ. Storage extraction is completed through structured, persistent TCP/IP database socket connections, ensuring zero port exposure to external public networks.

## 5. Architecture Trade Off Analysis Method Overview

The Architecture Trade-Off Analysis Method (ATAM) is a rigorous software engineering

evaluation framework used to assess a system's architectural decisions against clearly defined Quality Attribute Scenarios. Instead of reviewing abstract software code, ATAM evaluates high-level design topologies to determine how well they satisfy critical non-functional requirements such as scalability, maintainability, performance, and availability. Executing a simplified ATAM analysis allows the engineering team to uncover hidden architectural risks, identify key sensitivity points, and justify structural trade-offs based on clear data metrics.

## 6. ATAM Analysis for Scalability

### 6.1 Scalability Scenario

Scenario Definition: The restaurant experiences a sudden 4x surge in user traffic during a holiday peak hour. One hundred distinct tables simultaneously scan QR codes, generating high-frequency read requests for the digital menu catalog, while 40 tables submit multi-item shopping carts concurrently within a 5-minute window.

Target Metric: The system must process all incoming transactions without dropping connection sessions, keeping average menu loading latency under 2.0 seconds and maintaining a 0.00% error rate on active cart writes.

### 6.2 Monolithic Architecture and Scalability

A traditional, tightly coupled monolithic architecture handles scalability through vertical scaling (scaling up) by adding more CPU or RAM resources, or by duplicating the entire monolithic instance across a load balancer (horizontal scaling). While simple to manage, scaling a pure monolith is highly resource-inefficient. Because all modules reside within a single execution shell, a heavy traffic surge in tableside order submittals forces the entire server stack—including low-traffic services like user management or analytics—to scale up concurrently, leading to high infrastructure costs.

### 6.3 Microservices Architecture and Scalability

A fully distributed microservices architecture offers excellent scalability by allowing independent, fine-grained control over individual system nodes (horizontal scaling out). Under the holiday surge scenario, the *Menu Microservice* and *Order Tracking Service* can be scaled up across multiple container replicas without affecting other parts of the system. However, this approach introduces significant operational complexity, requiring

service discovery engines, distributed database synchronization, and network routing overhead that can increase network round-trip response times.

### 6.4 Scalability Decision in the System

FOODASH implements a highly optimized hybrid Modular Monolith combined with an asynchronous message broker..

Analysis: High-frequency menu browsing queries are offloaded by routing them through Nginx and an in-memory Redis cache layer. This configuration allows the primary application thread to handle a large volume of read requests with minimal resource usage. Concurrently, heavy database write mutations from cart submissions are deferred asynchronously via RabbitMQ queues. This design allows the application to achieve impressive scalability at a fraction of the operational complexity required by a full microservices layout.

## 7. ATAM Analysis for Availability

### 7.1 Availability Scenario

Scenario Definition: The external push gateway experiences a temporary network dropout or crashes while a customer is placing an order at a physical table.

Target Metric: The core tableside ordering and kitchen ticket printing systems must remain fully functional. The customer's order checkout journey must complete without interruption, and the system must guarantee that failed staff notification payloads are safely buffered and re-sent once network connectivity is restored.

### 7.2 Monolithic Architecture and Availability

In a traditional, non-modular monolithic system, availability is highly vulnerable due to the lack of failure isolation boundaries. If the internal notification component fails or experiences a thread-blocking error while trying to reach an external network gateway, it can cause the single shared execution process to hang. This block can quickly starve the server's thread pool, resulting in a cascading crash that brings down the entire system, preventing other tables from browsing menus or placing orders.

### 7.3 Microservices Architecture and Availability

Microservices architectures offer exceptional fault isolation because services are

completely decoupled and run in separate container runtimes. If the *Notification Microservice* crashes, the *Order Microservice* and *Menu Microservice* remain completely unaffected and continue processing transactions normally. The failure is completely isolated within its own service boundary, preventing system-wide downtime.

### 7.4 Availability Decision in the System

To maximize system availability without introducing excessive microservices infrastructure overhead, FOODASH uses an event-driven integration framework to separate critical and non-critical workflows.

Analysis: The *Order Processing Module* and the *Notification Module* communicate asynchronously using RabbitMQ message queues. When a customer submits an order, an immutable event payload is written directly to a durable queue, and the customer's request completes immediately. If the notification worker crashes, the messages simply queue up safely on disk within RabbitMQ. The core tableside ordering application continues to operate smoothly with zero downtime. Once the notification worker recovers, it processes the accumulated backlog, ensuring high availability and system resilience.

## 8. Discussion

The deployment layout and quality attribute analysis of the Training Curriculum Management System (UniMIS) in the provided reference report represents a practical compromise between architectural simplicity and long-term robustness. A fully distributed microservices mesh would maximize system scalability and availability, but it would significantly increase development friction, deployment costs, and operational monitoring complexity. Conversely, a standard monolithic structure would offer rapid development turnaround but pose major risks regarding performance bottlenecks and cascading system failures.

By adopting a hybrid architectural model—combining a strictly isolated Modular Monolithic backend engine with a standalone, asynchronous RabbitMQ message broker—the FOODASH system successfully balances production simplicity with enterprise-grade stability. This approach delivers exceptional low-latency performance and high availability during peak dining hours while keeping development, configuration, and monitoring highly manageable for the engineering team.

## 9. Conclusion

Lab 8 successfully provides a complete **Deployment View** and a detailed **Quality Attribute Analysis (ATAM)** for the QR Code-Based In-Table Food Ordering System (FOODASH). The containerized architecture modeled via Docker and Nginx establishes a highly secure, performant, and clean layout mapping software modules to isolated virtual hardware containers.

Furthermore, the formal ATAM evaluation confirms that the current hybrid design effectively manages the trade-offs between scalability and availability. By offloading read traffic via Redis and deferring complex transactional write mutations asynchronously through RabbitMQ, the system ensures sub-second tableside responsiveness and strong fault isolation while avoiding the high operational overhead of a fully distributed microservices network. This ensures that the FOODASH platform is highly stable, cost-effective, and fully prepared for future expansion into large-scale restaurant franchise operations.